

Build Your Own PDF-RAG Project — Step by Step (You Write The Code)

Rule for this guide: I explain WHAT each file should do, WHAT functions it needs, WHAT logic goes inside, and WHY. You write the actual Python. If you get stuck on syntax, come back and ask — but try first.

STEP 0 — Setup (do this before writing anything)

0.1 Create your project folder structure:

```
my_rag_project/
├── data/          ← put a sample PDF here
├── storage/       ← saved index will go here (empty for now)
├── src/
│   ├── __init__.py ← empty file, just makes src a package
│   ├── pdf_reader.py
│   ├── chunker.py
│   ├── embedder.py
│   ├── vector_store.py
│   ├── retriever.py
│   └── llm.py
├── ingest.py
├── chat.py
└── requirements.txt
```

Create every folder and empty file now. `__init__.py` can be completely empty — its only job is to tell Python "this folder is a package, you can import from it."

0.2 Set up a virtual environment (best practice, keeps your project's libraries separate from your system Python):

```
python -m venv venv
venv\Scripts\activate (on Windows)
```

0.3 Put this in `requirements.txt`, then install:

pypdf
sentence-transformers
faiss-cpu
numpy
ollama

Run: `pip install -r requirements.txt`

0.4 Install Ollama separately (it's not a pip package, it's a local app): download from ollama.com, then run `ollama pull llama3` in your terminal to download a free local model.

Why these libraries?

`pypdf` → reads text out of PDF files

`sentence-transformers` → turns text into number-vectors (embeddings), runs on your own machine, no API key needed

`faiss-cpu` → stores those vectors and searches them fast

`numpy` → the array math underneath embeddings/FAISS

`ollama` → runs a free LLM locally so you don't pay for API calls

STEP 1 — `src/pdf_reader.py`: Get text out of a PDF

Goal: One function that takes a file path, returns one big string of all the text in that PDF.

What to write:

```
python
def read_pdf(pdf_path):
    # 1. Open the PDF using pypdf's PdfReader
    # 2. Loop through every page
    # 3. Call .extract_text() on each page
    # 4. Glue all page texts together into one string
    # 5. Return that string
```

Hints:

```
from pypdf import PdfReader
```

```
reader = PdfReader(pdf_path)
```

`reader.pages` gives you a list of pages, loop with a `for` loop

Each `page.extract_text()` can return `None` if the page has no text (e.g. it's a scanned image) — handle that so your code doesn't crash (check `if text:` before adding it)

Test it yourself: write a throwaway line at the bottom temporarily: `print(read_pdf("data/sample.pdf")[:500])` — run the file directly, see if real text prints out. Delete that test line once it works.

STEP 2 — `src/chunker.py`: Split the big text into small pieces

Goal: One function that takes the full text string, returns a **list** of smaller text chunks.

Why we chunk: LLMs can't take unlimited text as input, and searching for "the most relevant paragraph" only works if you actually have separate paragraphs to compare — not one giant blob.

What to write:

```
python
def chunk_text(text, chunk_size=500, overlap=50):
    # 1. Decide your chunk size in characters (start with 500)
    # 2. Walk through the text in steps, cutting out chunk_size characters at a time
    # 3. IMPORTANT: overlap — each new chunk should start a little BEFORE
    #    where the last one ended, so you don't lose context at the boundary
    # 4. Collect all chunks into a list, return it
```

Hints:

A simple version uses a `while` loop with an index that moves forward by `(chunk_size - overlap)` each time, slicing the string like `text[start:start+chunk_size]`

Stop the loop when your index reaches the end of the text

Later improvement (not now, just note it): splitting on sentence boundaries instead of raw character counts gives cleaner chunks — come back to this in Step 10.

Test it yourself: `chunks = chunk_text(some_text)`, then `print(len(chunks))` and `print(chunks[0])` — check the sizes and overlap make sense.

STEP 3 — `src/embedder.py`: Turn text into vectors (numbers)

Goal: A class that loads an embedding model once, and can convert any list of text strings into a list of vectors.

What to write:

```
python
```

```

class Embedder:
    def __init__(self):
        # Load a pretrained sentence-transformers model
        # A good free small/fast one: "all-MiniLM-L6-v2"
        pass

    def encode(self, texts):
        # Call the model's own .encode() method on the list of texts
        # Return the result (it will be a numpy array of shape [num_texts, embedding_dim])
        pass

```

Hints:

```
from sentence_transformers import SentenceTransformer
```

```
Inside __init__: self.model = SentenceTransformer("all-MiniLM-L6-v2")
```

```
Inside encode: return self.model.encode(texts)
```

That's genuinely it for this file — sentence-transformers does the heavy lifting, your class is a thin wrapper. **Understand why you're wrapping it though:** if later you want to swap to a different embedding model (OpenAI's API, for example), you only change this one file — nothing else in your project needs to know or care.

Test it yourself:

```

python
e = Embedder()
vecs = e.encode(["hello world", "goodbye world"])
print(vecs.shape) # should print something like (2, 384)

```

STEP 4 — `src/vector_store.py`: Store vectors, search them, save/load from disk

Goal: A class wrapping a FAISS index — add vectors to it, search it, save it, load it.

What to write:

```

python

```

```

import faiss

class VectorStore:
    def __init__(self, dimension):
        # Create a FAISS index that expects vectors of this size
        # IndexFlatL2 is the simplest option — exact search, good for learning
        pass

    def add(self, embeddings):
        # Add the embeddings array into the index
        pass

    def save(self, path):
        # Use faiss.write_index() to save the index to a file
        pass

    @staticmethod
    def load(path):
        # Use faiss.read_index() to load a saved index back
        # This needs to return something usable — think about what
        # your other files expect when they call VectorStore.load(...)
        pass

```

Hints:

```

self.index = faiss.IndexFlatL2(dimension)

self.index.add(embeddings) — note: FAISS wants float32 numpy arrays; if you get a type error later, try
embeddings.astype('float32')

faiss.write_index(self.index, path)

faiss.read_index(path) returns a raw FAISS index object, not your VectorStore class — decide whether
load() should return just the raw index, or wrap it back into a VectorStore. (Look at how chat.py will use it
in Step 8, and make your choice consistent with that.)

```

Why IndexFlatL2 and not something fancier? It does exact brute-force search — slower at huge scale, but simple and 100% accurate, which is exactly right while you're learning. You can swap to a faster approximate index later once this works.

STEP 5 — src/retriever.py: Given a question-vector, find the closest chunks

Goal: One function that takes a query embedding + the FAISS index + your chunks list, returns the top matching chunk texts.

What to write:



```
python
def retrieve(query_embedding, index, chunks, top_k=3):
    # 1. FAISS search wants a 2D array, so reshape your single vector:
    #   query_embedding.reshape(1, -1)
    # 2. Call index.search(reshaped_vector, top_k)
    #   This returns TWO things: distances, and indices (the positions of best matches)
    # 3. Use those indices to pull out the actual text from your `chunks` list
    # 4. Return that list of matched chunk texts
```

Hints:

```
distances, indices = index.search(query_embedding.reshape(1, -1), top_k)
```

`indices` will look like `[[7, 2, 15]]` — a 2D array with one row. Loop through `indices[0]` to get the actual chunk positions.

```
[chunks[i] for i in indices[0]]
```

 gives you your final list

Think about this yourself: why do we need `distances` at all if we're just returning the `top_k` regardless? (Answer to think about: right now we don't use it, but later — Step 10 — you might want to filter out matches that are too far/irrelevant even if they're in the "top 3.")

STEP 6 — `src/llm.py`: Send question + context to the LLM, get an answer

Goal: One function that builds a prompt from the question and retrieved context, sends it to Ollama, returns the answer text.

What to write:



```
python
```

```
import ollama
```

```
def ask_llm(question, context):  
    # 1. Build a prompt string that includes BOTH the context and the question  
    #    Something like:  
    #    "Answer the question using ONLY the context below.  
    #    If the answer isn't in the context, say you don't know."  
    #  
    #    Context: {context}  
    #  
    #    Question: {question}"  
    # 2. Send this to ollama.chat() or ollama.generate()  
    # 3. Extract just the answer text from the response and return it
```

Hints:

```
response = ollama.chat(model="llama3", messages=[{"role": "user", "content": your_prompt}])
```

The answer text will be inside `response['message']['content']`

Why the "answer ONLY using context, say don't know if not there" instruction matters: without it, the LLM will happily make up plausible-sounding answers from its own general training knowledge instead of your PDF — this is hallucination, and that one instruction is your first (basic) defense against it.

STEP 7 — `ingest.py`: Wire everything together to BUILD the index

Goal: A script (not a function — just runs top to bottom) that processes a PDF once and saves the results.

What it needs to do, in order:

Import all your `src/` modules

Call `read_pdf()` on your sample PDF

Call `chunk_text()` on that text

Create an `Embedder()`, call `.encode()` on the chunks

Create a `VectorStore()` with the right dimension, `.add()` the embeddings

`.save()` the vector store to `storage/faiss_index.bin`

Pickle-save the chunks list to `storage/chunks.pkl`

Print something so you know it worked

You've already seen this exact pattern in the uploaded project — this is your chance to write it yourself without peeking, then compare.

STEP 8 — `chat.py`: Ask questions from the terminal

Goal: Load the saved index + chunks, loop asking for questions, print answers.

Load the vector store and chunks from disk (the reverse of Step 7's saving)

Create an `Embedder()`

`while True:` loop — take input, break on `"exit"`

Encode the question, retrieve top chunks, join them into a context string

Call `ask_llm()`, print the answer

STEP 9 — (Optional once Step 8 works) `app.py`: Streamlit version

Only attempt this after `chat.py` fully works — a browser UI on top of a broken pipeline just hides bugs. Once ready, come back and I'll walk you through the Streamlit-specific parts (file upload, `session_state`) the same step-by-step way.

STEP 10 — Test it like a real engineer would

Once everything runs, don't stop — actually try to break it:

Ask a question whose answer is clearly IN the PDF → check it's correct

Ask something totally unrelated to the PDF → does it say "I don't know" or does it hallucinate an answer?

Ask about something using different words than the PDF uses (paraphrase it) → does retrieval still find it?

Note down what breaks — that becomes your honest "here's the weakest part" answer for your mentor, and your task list for Week 3-4 of your roadmap (hybrid search, better chunking, citations).

If You Get Stuck

Come back with the SPECIFIC error message or the SPECIFIC function you're unsure about — not "it doesn't work." Paste the error, tell me which step you're on, and I'll help you debug it without just handing you the fix outright unless you want the full working line to compare against your own attempt.